

CSS Bootcamp, Day 5

AI Workflows

24 September 2026

Agenda

Today we're going to talk about something you probably use everyday, whether you know it or not: large language models (LLMs). We're **NOT** going to talk about how to use LLMs as prediction or classification machines—that's a subject for later. Instead, we're going to focus on how to incorporate LLMs into your coding workflow with **coding agents**.

Our Goals for Today:

1. **OpenCode & Agents:** What agentic workflows are and why they are useful.
2. **Live Walk-through:** Debugging and writing files with the MiMo agent.
3. **Data Sovereignty:** How to connect OpenCode to local hardware via Ollama.
4. **Afternoon Challenge:** AI-assisted debugging of a broken data pipeline.

Generative AI & Large Language Models

Traditional machine learning in the social sciences usually focuses on **predictive or analytical tasks** (e.g., OLS regression, classification, or clustering).

Generative AI represents a paradigm shift. Instead of just analyzing existing data, these models learn the underlying statistical distribution of their training data to *create entirely new data* (text, images, or code).

A **Large Language Model (LLM)** is a specific type of generative AI trained on vast corpora of text. By mapping the statistical relationships between words across billions of parameters, an LLM learns to predict the most probable next "token" (a word or sub-word) in a sequence.

The Transformer Architecture

Modern LLMs are built on the **Transformer** architecture, introduced by Vaswani et al. [1]

Transformers use a mechanism called **Self-Attention**.

Self-attention allows the model to look at an *entire* sequence of text simultaneously. It weighs the importance of each word relative to every other word, allowing the model to capture deep contextual meaning and long-range dependencies.

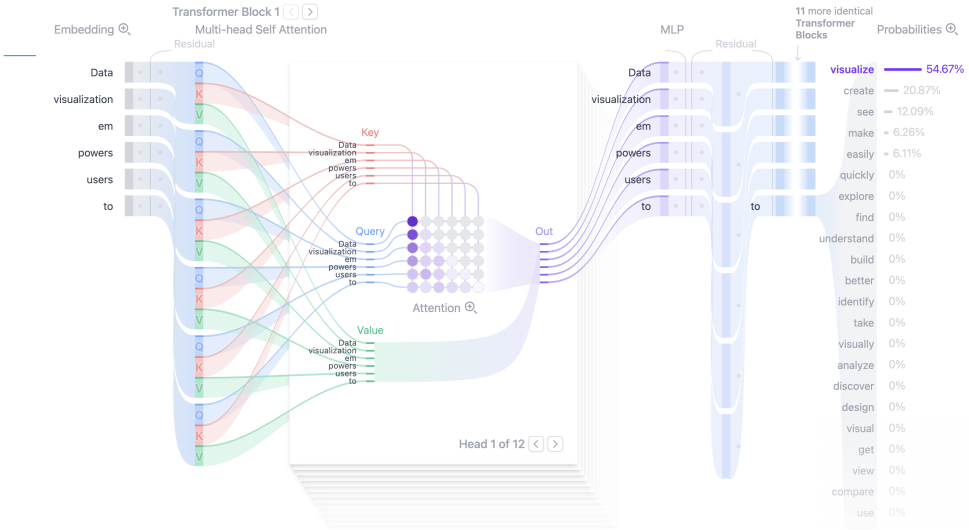


Figure: Interactive Transformer. From here

Encoders vs. Decoders

The original Transformer architecture has two halves. Today, models usually isolate and specialize in just one:

1. The Encoder (e.g., BERT)

- ▶ Reads the entire input sequence simultaneously (bidirectional).
- ▶ Excellent for *understanding* text: classification, sentiment analysis, and generating static word embeddings.

2. The Decoder (e.g., GPT, Llama, Claude)

- ▶ Generates text left-to-right, predicting one token at a time (auto-regressive).
- ▶ Excellent for *generating* text: answering questions, translating, and writing code.
- ▶ LLMs as you understand them are decoder-only models.

What is OpenCode?

Most AI tools (like ChatGPT) exist in a browser tab, completely blind to your work. We want our AI to be aware of our scripts, errors, and file directories.

The OpenCode Extension in Positron:

- ▶ Brings an AI chat interface directly into your session.
- ▶ **Context Injection:** It can read your active file or fix a broken function in your script.
- ▶ **Autonomous Tool Use:** It is not just a chat bot—it is an *agent*. It can read your repository, execute terminal commands, and write code directly to your files without you copying and pasting.

Installing and Configuring OpenCode

Getting started in Positron is straightforward:

1. Open the **Extensions** tab in Positron's left sidebar.
2. Search for **OpenCode** and click Install.
3. Open the OpenCode panel. Type `/models` in the OpenCode textbox to see the available LLMs.
4. For today's class, select **MiMo-V2.6-Flash Free**.

Note: MiMo is a pre-shipped, cloud-hosted frontier model. It is incredibly powerful and handles tool execution.

Walk-through: The Setup

Let's look at a classic beginner mistake in R. Suppose we want to filter a dataset and then plot it. Open a blank script and paste this:

```
library(tidyverse)

# We want to filter for 4-cylinder cars and plot a scatter
mtcars |>
  filter(cyl == 4) |>
  ggplot(aes(x = wt, y = mpg)) |>
  geom_point()
```

An error will print in the console.

Walk-through: Prompting the Agent

Copy this entire chunk of code in Positron.

We don't just want an answer; we want the agent to do the work. Enter this exact prompt:

"I am getting an error trying to pipe a filtered dataframe directly into a ggplot object. Explain why this failed syntactically, and then write the corrected script into a new file called fixed_plot.R."

Paste the copied code directly under the prompt, then press *Enter*.

Walk-through: The Agentic Result

Notice what happens next:

- ▶ **Reasoning:** MiMo explains that the native pipe (`|>`) passes dataframes, but `ggplot2` layers are added together using the `+` operator.
- ▶ **Result:** Look at your file explorer! A new file named `fixed_plot.R` has been automatically created in your directory containing the patched code.

Another Walk-Through

Let's try a more substantial example. Remember the messy ESS data extract you cleaned on bootcamp day #1? Let's see how good MiMo V2.6 Flash is at doing it for us.

"Write a Quarto document that loads the `ess_messy_extract.csv` dataset in my `data/` subfolder. This is a messy version of the ESS round 11 for France. Go through the variables and clean them. Write a Quarto document that details the step by step instructions for cleaning the csv, from loading in the data to saving the cleaned data. Save the clean data as `data/ess_cleaned.csv`. Save the Qmd as `ess_cleaning.qmd`. Render the Quarto as an html document. I should see a `ess_cleaning.html` file."

MiMo is fantastic, but as computational social scientists, we often work with sensitive data. Sending code to cloud APIs isn't always legally or ethically viable.

The Open Science Standard:

- ▶ You can keep computation completely local by pointing OpenCode to an **Ollama** instance running on your own hardware.
- ▶ This bypasses external APIs entirely, guaranteeing data sovereignty.
- ▶ *While we won't run this live in class due to laptop hardware constraints, you should know how to configure this for your lab workstations.*

Fetching Local Models

If you have a powerful workstation, you can pull permissive open-weights models via your terminal.

Recommended Heavy-Duty Models for Coding:

```
# Qwen 2.5 Coder (32 Billion Parameters)
# Arguably the best open-weights coding model currently
  available.
ollama pull qwen2.5-coder:32b

# GPT-OSS (20 Billion Parameters)
# Highly stable for complex reasoning tasks.
ollama pull gpt-oss:20b
```

Configuring OpenCode for Local Models

To link these local models to Positron, you create a configuration file at `~/.config/opencode/opencode.json` to tell OpenCode to look at your localhost instead of the cloud:

```
{
  "provider": {
    "ollama": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "Ollama (local)",
      "options": { "baseUrl": "http://localhost:11434/v1" },
      "models": {
        "qwen2.5-coder:32b": { "name": "Qwen 2.5 (32B)" },
        "gpt-oss:20b": { "name": "GPT-OSS (20B)" }
      }
    }
  }
}
```

Once saved, these models appear right in your OpenCode dropdown.

Why Larger Models for Agents?

You might wonder why we suggest 20B or 32B parameter models for Ollama, rather than the lightweight 7B/8B models that easily run on laptops.

The Agentic Overhead:

- ▶ OpenCode does not just send your prompt; it wraps it in massive, hidden JSON instructions defining how the agent should use tools (like `write`, `read`, `bash`).
- ▶ Smaller models (under 10B parameters) lack the "brain space" to juggle these system instructions alongside your coding problem. They often hallucinate non-existent tools or get trapped in infinite loops.
- ▶ True agentic behavior requires massive parameter counts (like MiMo, or local 30B+ models) to reliably separate tool-calling logic from conversational output.

References I

- [1] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).



BREAK FOR LUNCH



Center for Research in Economics and Statistics
CREST UMR 9194 - 5 avenue Henry Le Chatelier
91120 Palaiseau

Group Application: The Broken Pipeline Challenge

Follow the instructions here: https://bit.ly/css_bootcamp_day5.